Cognitive Modeling

Complete Lecture Notes

Week 11: Monte Carlo Methods and the Metropolis–Hastings Algorithm

Contents

1 Parameter Estimation with MCMC 1

1.1 Week 11: Monte Carlo Methods and the Metropolis–Hastings Algorithm 1

1.1.1 Key terms 1

1.1.2 Learning outcomes 2

1.1.3 Motivation & big picture 3

1.1.4 Lecture notes 3

1.1.5 Worked example 7

1.1.6 Lab (2 hours) 9

1.1.7 Connection to AI & LLMs 12

1.1.8 Common mistakes & misconceptions 12

1.1.9 Check your understanding 13

1.1.10 Assignment 14

1.1.11 Further reading 14

Chapter 1

Parameter Estimation with MCMC

1.1 Week 11: Monte Carlo Methods and the Metropolis–Hastings Algorithm

Chapter 5: Parameter estimation with data-driven methods (MCMC) · Lecture 1 h · Lab 2 h Running data: the Week 2 forgetting data with the Week 10 priors, so that we can check the sampler against the grid posterior we already have.

1.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
Monte Carlo	Monte Carlo (مونت‌کارلو)	تقریب با میانگین نمونه‌های تصادفی
Metropolis algorithm	الگوریتم Metropolis	پیشنهاد یک حرکت و پذیرش یا رد آن
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآورد / برآوردگر	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
Markov chain Monte Carlo (MCMC)	زنجیره‌ی مارکوف Monte Carlo	نمونه‌گیری از پسین با یک زنجیره‌ی تصادفی
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
sampler	نمونه‌گیر (sampler)	الگوریتمی که از توزیع نمونه می‌گیرد

English	فارسی	توضیح کوتاه
grid approximation	تقریب شبکه‌ای (grid)	محاسبه‌ی پسین روی شبکه‌ای از مقادیر
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
chain	زنجیره	دنباله‌ی نمونه‌های MCMC
proposal	پیشنهاد (proposal)	مقدار جدیدی که الگوریتم امتحان می‌کند
acceptance rate	نرخ پذیرش	نسبت پیشنهادهای پذیرفته‌شده
trace plot	نمودار trace	مقدار نمونه‌ها در برابر شماره‌ی تکرار
autocorrelation	خودهمبستگی	وابستگی نمونه‌های پشت سر هم
burn-in / warm-up	دوره‌ی گرم‌شدن (burn-in)	نمونه‌های اولیه که دور ریخته می‌شوند
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
starting point	نقطه‌ی شروع	سوگیری اولیه به سمت یکی از پاسخ‌ها
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
self-consistency	self-consistency (رأی اکثریت)	چند بار نمونه‌گیری جواب و انتخاب رأی اکثریت
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
MCSE	خطای استاندارد Monte Carlo	خطای شبیه‌سازی در میانگین پسین

1.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Explain why posterior quantities are integrals, and how Monte Carlo sampling turns integrals into averages, with an error that shrinks like $1/\sqrt{S}$.
2. Describe the logic of Markov chain Monte Carlo: a chain whose long-run distribution is the posterior, using only the *unnormalized* posterior.
3. Implement the Metropolis (and Metropolis–Hastings) algorithm from scratch for a one- and a two-parameter cognitive model.
4. Tune the proposal distribution using the acceptance rate, trace plots and autocorrelation, and identify burn-in.
5. Compute posterior summaries and derived quantities (e.g., a half-life) from samples, and validate a sampler against a known answer.

1.1.3 Motivation & big picture

Everything we want from a posterior is an integral: the posterior mean $\int \theta\, p(\theta \mid \mathrm{y})\, d\theta$, a marginal posterior (integrating out other parameters), $P(d' > 1 \mid \mathrm{y})$, the posterior predictive distribution, the marginal likelihood. In Weeks 9–10 we did these integrals on grids. That worked for two parameters. A hierarchical SDT model for 20 participants (Week 13) has more than 40 parameters; a grid with only 10 points per parameter would need 10^{40} evaluations.

Monte Carlo methods replace integration by sampling: if we could draw many values from the posterior, any integral becomes an average over the draws, and the accuracy depends on the number of draws, not on the number of parameters. The problem is that we cannot sample from a posterior directly; we usually cannot even compute its normalizing constant.

Markov chain Monte Carlo (MCMC) solves this with a beautifully simple trick: a random walk through parameter space that moves toward more probable values more often than toward less probable ones, in exactly the right proportions. After a while, the places the walker visits are draws from the posterior. The best-known version, the Metropolis algorithm, fits in 15 lines of Python, and you will write it today before using any library.

Where is this used?

- All modern Bayesian software (PyMC, Stan, JAGS, BUGS) is built on MCMC. Every hierarchical model in recent psychology and neuroscience papers was fitted with some version of it.
- Physics and chemistry (where Metropolis was invented in 1953, to simulate particles), epidemiology (estimating infection parameters), phylogenetics (reconstructing evolutionary trees) and finance all rely on MCMC.
- Cognitive science also uses sampling as a *theory of the mind*: some researchers propose that people approximate Bayesian inference by mentally sampling a few possibilities, which explains certain biases.
- AI: sampling answers from a language model is Monte Carlo (see the AI & LLMs section).

1.1.4 Lecture notes

In plain words Every question we ask a posterior ("what is the average value?", "what is the probability that $d' > 1$?") is an integral, and grids cannot compute integrals over many parameters. Monte Carlo replaces the integral with an average: if we had many random values *drawn from* the posterior, the average of those values would be the posterior mean, the fraction above 1 would be $P(d' > 1)$, and so on. The difficulty is getting those draws. The Metropolis algorithm gets them with a clever random walk. Stand at some parameter value, propose a small random step, and compare how plausible (likelihood × prior) the new spot is. If it is more plausible, move there. If it is less plausible, move there only *sometimes*, with probability equal to the ratio of plausibilities; otherwise stay where you are and record the current value again. Run this long enough and the places you have visited are

a sample from the posterior. Remarkably, you never need the hard normalizing constant, because only ratios are used.

Suggested timing (60 min): Monte Carlo integration 12 · Markov chains 8 · the Metropolis algorithm 18 · Metropolis–Hastings 5 · tuning, burn-in and autocorrelation 12 · preview 5.

1. Monte Carlo integration

If $\theta^{(1)}, \ldots, \theta^{(S)}$ are independent draws from $p(\theta \mid \mathrm{y})$, then for any function g:

$$\mathbb{E}[g(\theta) \mid \mathrm{y}] = \int g(\theta)\, p(\theta \mid \mathrm{y})\, d\theta \;\approx\; \frac{1}{S} \sum_{s=1}^{S} g\big(\theta^{(s)}\big).$$

Want	Choose $g(\theta)$	Monte Carlo estimate
Posterior mean	θ	Average of the draws
$P(\theta > 1 \mid \mathrm{y})$	indicator $\mathbb{1}[\theta > 1]$	Proportion of draws above 1
Posterior of a derived quantity, e.g., half-life $2^{1/d} - 1$	$g(d)$	Apply g to each draw, then summarize
Credible interval	—	Percentiles (or HDI) of the draws
Marginal posterior of θ_1	—	Simply ignore the other columns

The Monte Carlo standard error of the mean is $\mathrm{SD}(g)/\sqrt{S}$ for independent draws, independent of the number of parameters. That is why sampling beats grids in high dimensions.

Reading the math

- $\mathbb{E}[g(\theta) \mid \mathrm{y}] = \int g(\theta)\, p(\theta \mid \mathrm{y})\, d\theta$: the posterior average of some quantity $g(\theta)$. Choosing g chooses the question (see the table).
- $\approx \frac{1}{S} \sum_s g(\theta^{(s)})$: replace the integral by the plain average over S draws $\theta^{(1)}, \ldots, \theta^{(S)}$. The superscript (s) is the draw number.
- Error: $\mathrm{SD}(g)/\sqrt{S}$. To halve the error you need four times as many draws, whatever the number of parameters.
- Example: 10,000 draws from Beta$(8, 4)$; 9,000 of them are above 0.5, so $P(\theta > 0.5 \mid \mathrm{y}) \approx 0.90$, with Monte Carlo error $\sqrt{0.9 \times 0.1/10000} = 0.003$.

2. Markov chains in one paragraph

A Markov chain is a sequence $\theta^{(1)}, \theta^{(2)}, \ldots$ in which each value is generated from the previous one only, by a fixed transition rule. Under mild conditions, a chain "forgets" its starting point and settles into a stationary distribution: the long-run proportion of time it spends in each region. MCMC

constructs a transition rule whose stationary distribution is the posterior. Two consequences: - The early part of the chain still reflects the starting point. It is discarded as burn-in (warm-up). - Successive draws are correlated, so S correlated draws carry less information than S independent ones (the effective sample size, Week 12).

3. The Metropolis algorithm

Let $\tilde{p}(\theta) = p(\mathrm{y} \mid \theta)\, p(\theta)$ be the unnormalized posterior (likelihood × prior). Choose a symmetric proposal distribution, usually a normal random walk $\theta' \sim \mathcal{N}(\theta, \sigma_{\text{prop}})$.

1. Start at some $\theta^{(0)}$.
2. For $s = 1, 2, \ldots, S$:
   - Propose $\theta' \sim \mathcal{N}(\theta^{(s-1)}, \sigma_{\text{prop}})$.
   - Compute the acceptance probability
   $$\alpha_{\text{acc}} = \min\left(1,\ \frac{\tilde{p}(\theta')}{\tilde{p}(\theta^{(s-1)})}\right).$$
   - Draw $u \sim \text{Uniform}(0, 1)$. If $u < \alpha_{\text{acc}}$, accept: $\theta^{(s)} = \theta'$. Otherwise reject: $\theta^{(s)} = \theta^{(s-1)}$ (the current value is *repeated* in the chain).

Why it works intuitively: uphill moves are always accepted; downhill moves are accepted with probability equal to the density ratio. So the chain spends time in each region in proportion to its posterior probability.

Why only the unnormalized posterior is needed: the evidence $p(\mathrm{y})$ appears in both numerator and denominator of the ratio and cancels. This is the key to MCMC's usefulness.

Practical details: - Work in logs: accept if $\ln u < \ln \tilde{p}(\theta') - \ln \tilde{p}(\theta)$. - Bounded parameters: if a proposal falls outside the support (e.g., $A > 1$), its prior is 0, so $\ln \tilde{p} = -\infty$ and the proposal is automatically rejected. (Alternatively, sample on a transformed, unbounded scale, as in Week 3; then the Jacobian must be added to the log posterior.) - Multiple parameters: propose all parameters at once with a vector of proposal SDs, or update one parameter at a time (which leads to Gibbs sampling, Week 12).

> Advanced: detailed balance. A transition rule $T(\theta \to \theta')$ leaves π stationary if $\pi(\theta)\, T(\theta \to \theta') = \pi(\theta')\, T(\theta' \to \theta)$ for all pairs. Check it for Metropolis with $\tilde{p}(\theta') < \tilde{p}(\theta)$: the left side is $\pi(\theta)\, q(\theta' \mid \theta)\, \tilde{p}(\theta')/\tilde{p}(\theta)$ and the right side is $\pi(\theta')\, q(\theta \mid \theta') \cdot 1$. With symmetric q and $\pi \propto \tilde{p}$, the two are equal.

Reading the math

- $\tilde{p}(\theta) = p(\mathrm{y} \mid \theta)\, p(\theta)$: likelihood × prior, *without* dividing by the evidence. The tilde (~) marks "unnormalized".

- $\alpha_{\text{acc}} = \min(1, \tilde{p}(\theta')/\tilde{p}(\theta))$: if the proposal is more plausible, the ratio is > 1 and min caps it at 1 (always accept). If it is half as plausible, accept with probability 0.5.
- "Draw $u \sim \text{Uniform}(0, 1)$; accept if $u < \alpha$" is simply a way to make an event happen with probability α (a biased coin).
- Worked example, step 3: the proposal 0.95 is only 0.052 times as plausible as 0.80; $u = 0.30 > 0.052$, so it is rejected and 0.80 is written into the chain a second time.

4. Metropolis–Hastings: asymmetric proposals

If the proposal is not symmetric, $q(\theta' \mid \theta) \neq q(\theta \mid \theta')$ (e.g., a log-normal proposal for a positive rate), the acceptance probability needs a correction:

$$\alpha_{\text{acc}} = \min\left(1,\ \frac{\tilde{p}(\theta')\, q(\theta \mid \theta')}{\tilde{p}(\theta)\, q(\theta' \mid \theta)}\right).$$

Metropolis is the special case where the q terms cancel. Gibbs sampling (Week 12) is another special case, in which every proposal is accepted.

5. Tuning and diagnosing a sampler

The proposal SD σ_{prop} controls efficiency:

σ_{prop}	Acceptance rate	Behavior	Trace plot looks like
Too small	Very high (> 0.9)	Tiny steps; explores slowly; strong autocorrelation	A slowly wandering line
Too large	Very low (< 0.1)	Most proposals land in low-probability regions and are rejected; the chain sticks	Flat steps with long plateaus
About right	≈ 0.2–0.5	Good mixing	A "fuzzy caterpillar"

Classic theoretical results suggest an optimal acceptance rate of about 0.44 for one parameter, falling toward about 0.23 for many parameters. Treat these as targets, not laws. Tune σ_{prop} during a preliminary run (not during the run you keep). Good proposal SDs are of the order of the posterior SDs, and correlated posteriors need correlated (or reparameterized) proposals.

Burn-in. Discard the initial part of the chain before it reaches the high-probability region; the trace plot shows where. Thinning (keeping every m-th draw) reduces storage but throws away information. It is not needed for correct inference.

Autocorrelation. The lag-h autocorrelation is the correlation between $\theta^{(s)}$ and $\theta^{(s+h)}$. High autocorrelation means slow exploration and fewer effectively independent draws.

Always validate a new sampler on a problem with a known answer (here, the conjugate beta-binomial and the Week 10 grid).

For an engineer

- Monte Carlo integration is what you do when you estimate a BER or an outage probability by simulation: average an indicator over many random realizations. The $1/\sqrt{S}$ error law is the same, and it is why low error rates need very long simulations.
- An MCMC chain is a Markov process whose stationary distribution is designed to be the posterior; burn-in is the transient, the stationary part is the steady state.
- Autocorrelation of the chain is exactly the autocorrelation of a random process. Correlated samples carry less information, just as correlated noise samples average out more slowly; Week 12 turns this into the "effective sample size".
- Proposal tuning is a step-size problem like choosing the step size of an adaptive filter: too small converges slowly, too large becomes unstable (here: most steps are rejected).
- The difference from ordinary simulation: we cannot generate independent samples from the target directly, so we build a process that converges to it.

For the instructor

- Walk the chain on the board. Draw the curve $\theta^7(1-\theta)^3$ and run the four steps of the worked example with a student rolling the "u" values (a random number app works). The rejected step that *repeats* 0.80 is the key moment; ask why it must be recorded.
- An analogy that works: a tourist exploring a city at night who prefers lit streets. Brighter street: always go. Darker street: go only sometimes. After a long walk, the time spent on each street is proportional to its brightness.
- Show the three trace plots (too small, good, too large) from lab Part D side by side. Students learn to read trace plots in five minutes this way.
- Typical questions.
  - *"Why not just use a fine grid?"* With 40 parameters, a grid is impossible; sampling cost does not grow that way.
  - *"How do we know the chain has converged?"* That is Week 12 (multiple chains, $\hat{R}$).
  - *"Why throw away burn-in?"* Early draws still reflect the arbitrary starting point, not the posterior.

1.1.5 Worked example

Model: recall $y = 7$ of $n = 10$ with a flat prior, so $\tilde{p}(\theta) = \theta^7(1-\theta)^3$ (constants dropped). Exact posterior: Beta$(8, 4)$, mean 0.667. Proposal: $\theta' = \theta + \mathcal{N}(0, 0.1)$.

Step	Current θ	Proposal θ'	$\tilde{p}(\theta)$	$\tilde{p}(\theta')$	Ratio	u	Decision	New θ
1	0.50	0.58	0.000977	0.001636	1.675	0.42	accept (uphill)	0.58
2	0.58	0.80	0.001636	0.001678	1.026	0.71	accept (uphill)	0.80
3	0.80	0.95	0.001678	0.000087	0.052	0.30	reject: $u > 0.052$	0.80
4	0.80	0.66	0.001678	0.002144	1.278	0.88	accept (uphill)	0.66

Chain so far: 0.50, 0.58, 0.80, 0.80, 0.66. The rejected step *repeats* 0.80 in the chain. That is essential: repeated values are how the chain records that 0.80 is more probable than 0.95. Dropping them would bias the sample.

Interpretation. In four steps the chain moved from the start into the region of high posterior density (0.6–0.8) and refused a jump into the tail. Over thousands of steps, the proportion of time spent near each value converges to Beta$(8, 4)$.

1.1.6 Lab (2 hours)

Goals

1. Monte Carlo integration with independent draws: see the $1/\sqrt{S}$ error.
2. Write a Metropolis sampler and validate it on the beta-binomial model.
3. Sample the posterior of the power forgetting model, compare with the Week 10 grid, and compute a derived quantity.
4. Tune the proposal: compare too-small, good and too-large proposal SDs with trace plots, acceptance rates and autocorrelation.

Timing: Part A 15 min · Part B 25 min · Part C 35 min · Part D 30 min · Part E 15 min.

Setup

Code · کد: `code/week11.py`, cell 1 · سلول 1

Part A: Monte Carlo integration (15 min)

Code · کد: `code/week11.py`, cell 2 · سلول 2

Expected output:

```
Exact: mean = 0.5926, P(theta > 0.65) = 0.1967
S =    100: mean = 0.5865 (MC s.e. 0.0065), P(theta > 0.65) = 0.1800
S =   1000: mean = 0.5935 (MC s.e. 0.0021), P(theta > 0.65) = 0.2160
S =  10000: mean = 0.5937 (MC s.e. 0.0007), P(theta > 0.65) = 0.2022
S = 100000: mean = 0.5927 (MC s.e. 0.0002), P(theta > 0.65) = 0.1971
```

Ten times more draws gives about $\sqrt{10} \approx 3.2$ times less Monte Carlo error.

Part B: a Metropolis sampler, validated (25 min)

Code · کد: `code/week11.py`, cell 3 · سلول 3

Expected output:

```
Acceptance rate: 0.59
Metropolis: mean = 0.5930, SD = 0.0660, 95% interval = [0.458 0.718]
Exact:      mean = 0.5926, SD = 0.0663, 95% interval = [0.46  0.719]
```

Part C: Metropolis for the power forgetting model (35 min)

Same priors as in Week 10: $A \sim \text{Beta}(8, 2)$, $d \sim \text{Gamma}(2, \text{scale } 0.3)$.

Code · کد: `code/week11.py`, cell 4 · سلول 4

Expected output:

```
Acceptance rate: 0.49
    mean     SD   2.5%  97.5%
A  0.868  0.052  0.762   0.96
d  0.418  0.048  0.325   0.51
Week 10 grid posterior means for comparison: A = 0.867, d = 0.417
```

Derived quantities come for free. Compute the posterior of the half-life $t_{1/2} = 2^{1/d} - 1$ (Week 2) and the predicted recall after 60 days:

Code · کد: `code/week11.py`, cell 5 · سلول 5

Expected output:

```
half-life (days)       mean = 4.493, 95% interval = [2.889 7.443]
P(recall) at 60 days   mean = 0.158, 95% interval = [0.113 0.211]
P(half-life < 3 days | data) = 0.039
```

Part D: tuning the proposal (30 min)

Code · کد: `code/week11.py`, cell 6 · سلول 6

Expected output:

```
too small    proposal SD = [0.002, 0.002]: acceptance = 0.97, lag-10
↪  autocorrelation of d = 0.98, mean of d = 0.473
about right  proposal SD = [0.04, 0.05]: acceptance = 0.50, lag-10
↪  autocorrelation of d = 0.26, mean of d = 0.416
too large    proposal SD = [0.5, 1.5]: acceptance = 0.00, lag-10
↪  autocorrelation of d = 0.94, mean of d = 0.420
```

What to notice: the "too small" chain accepts almost everything but crawls (a slowly wandering trace with autocorrelation near 1 even at lag 60); the "too large" chain rejects almost everything and sits still for long stretches (flat steps). Both give unreliable estimates from 10 000 iterations: the "too small" chain's mean of d (0.473) is far from the true posterior mean (0.417), and the "too large" chain's mean happens to be close only because it barely moved from its starting value (0.42). The well-tuned chain looks like a fuzzy caterpillar and its autocorrelation decays quickly.

Part E: interpret (15 min)

Answer in your notebook: 1. Do the Metropolis posterior summaries for (A, d) agree with the Week 10 grid? What does that agreement establish? 2. How would you report the half-life of this participant's memory, with uncertainty, in one sentence? 3. Why must rejected proposals be recorded as repeated values in the chain?

Exercises (increasing difficulty)

Exercise 1 $\star$. Using the Part B sampler, estimate $P(\theta > 0.65 \mid \text{y})$ for 30/50 and compare with the exact value from Part A. How many iterations do you need to get within ± 0.01?

Exercise 2 $\star$. Run the Part C sampler from four very different starting points, e.g., $(0.3, 0.05)$, $(0.99, 2.5)$, $(0.5, 1.0)$ and $(0.9, 0.1)$. Plot all traces together. After how many iterations do they become indistinguishable? (This is the idea behind R-hat, Week 12.)

Exercise 3 $\star\star$ (exponential model). Write `log_post_exp` with the Week 10 priors, sample it, and compare the posterior with the grid. Then compute the posterior predictive distribution from your samples and reproduce the Week 10 PPC plot.

Exercise 4 $\star\star$ (Metropolis–Hastings with an asymmetric proposal). Sample d with a log-normal random-walk proposal, $\ln d' \sim \mathcal{N}(\ln d, 0.1)$, which never proposes negative values. Derive the Hastings correction $q(d \mid d')/q(d' \mid d) = d'/d$ and include it. Show that omitting the correction biases the posterior.

Exercise 5 $\star\star\star$ (Advanced: adaptive tuning). Write a warm-up phase that adjusts the proposal SDs every 500 iterations to push the acceptance rate toward 0.3 (e.g., multiply the SD by 1.2 if the rate is too high, by 0.8 if too low), then freezes them for the main run. Why must the adaptation stop before the samples you keep?

Lab check-off

Show: (1) the validation of your sampler against the exact Beta posterior (Part B), (2) the trace plots and summary for the power model with the comparison to the grid (Part C), and (3) the tuning figure with acceptance rates (Part D).

1.1.7 Connection to AI & LLMs

Sampling answers from an LLM is Monte Carlo. Because an LLM samples its output (Week 1 box), asking the same question several times gives different answers. A popular trick called self-consistency samples k answers and takes the majority vote [Wang et al., 2023, ICLR]. Does it help, and how much? This is a Monte Carlo question: we cannot compute the majority-vote accuracy easily in closed form, but we can simulate it and report the Monte Carlo standard error, as in this week's lab.

Code · کد: `code/week11.py`, cell 7 · سلول 7

Expected output:

```
k =  1 samples: majority-vote accuracy = 0.406 (MCSE 0.003)
k =  5 samples: majority-vote accuracy = 0.506 (MCSE 0.004)
k = 15 samples: majority-vote accuracy = 0.671 (MCSE 0.003)
k = 45 samples: majority-vote accuracy = 0.873 (MCSE 0.002)
```

Try it (no API needed). - ⋆ Change `p_answer` so that one *wrong* answer is the most likely (e.g., `[0.30,0.35,0.20,0.15]`). What does majority voting do now? Why? - ⋆⋆ How many simulated questions (`n_sim`) do you need for an MCSE of 0.001? Check your answer with the formula $\sqrt{p(1-p)/n}$. - ⋆⋆ Connection to cognition: the "wisdom of the crowd within" idea says that averaging a person's repeated guesses improves accuracy. Which assumption of this simulation (independent samples) is unlikely to hold for repeated human guesses, or for an LLM at low temperature?

1.1.8 Common mistakes & misconceptions

1. Dropping rejected steps instead of repeating the current value. This biases the sample toward the tails.
2. Using the normalized posterior. Not needed and usually impossible; the evidence cancels in the ratio.
3. Computing the ratio without logs. Likelihoods underflow. Compare log densities.

4. Forgetting the prior support. Proposals outside the support must get $\ln \tilde{p} = -\infty$, otherwise the likelihood may be evaluated at impossible values (e.g., negative probabilities produce `nan`).
5. Judging the sampler only by acceptance rate. A 30% rate does not guarantee convergence. Look at traces and autocorrelation, and run several chains (Week 12).
6. Keeping the burn-in or tuning the proposal during the samples you keep. Both bias the results.
7. "More iterations always fix it." If the chain is stuck in one mode, or the proposal is badly scaled, more iterations of the same chain may not help in practical time. Diagnose first.
8. Forgetting the Hastings correction with asymmetric proposals.

1.1.9 Check your understanding

1. Why is the Monte Carlo error independent of the number of parameters, while grid approximation is not? Answer The MC estimate is an average of S draws, and its standard error is $\text{SD}/\sqrt{S}$ regardless of the dimension of θ. A grid must cover the whole parameter space, so its size grows exponentially with the number of parameters.

2. In the Metropolis algorithm, why do we not need the evidence $p(\text{y})$? Answer Acceptance depends only on the *ratio* of posterior densities at the proposed and current values; $p(\text{y})$ appears in both and cancels.

3. Current $\tilde{p}(\theta) = 0.004$, proposed $\tilde{p}(\theta') = 0.001$, and $u = 0.18$. Accept or reject? What if $u = 0.31$? Answer The ratio is 0.25. With $u = 0.18 < 0.25$, accept; with $u = 0.31 > 0.25$, reject and repeat the current value.

4. A chain has acceptance rate 0.97. What is probably wrong, and how does its trace look? Answer The proposal SD is too small: almost every tiny step is accepted, the chain explores very slowly, and the trace looks like a slowly wandering line with high autocorrelation. Increase the proposal SD.

5. What is burn-in, and how do you decide its length? Answer The initial iterations, before the chain has reached the high-probability region, which still reflect the arbitrary starting value. Choose the length from trace plots (and from several chains started at different points), and discard those iterations.

6. When is the Hastings correction needed? Give an example. Answer When the proposal is asymmetric, $q(\theta' \mid \theta) \neq q(\theta \mid \theta')$, e.g., a log-normal proposal for a positive rate, or a proposal that depends on the current value's position in a bounded interval.

7. How do you obtain the posterior of the half-life $t_{1/2} = 2^{1/d} - 1$ from MCMC samples of d? Answer Transform every sample of d into a half-life and summarize the transformed samples (mean, percentiles, HDI). No extra derivation is needed; this is one of the big practical advantages of sampling.

1.1.10 Assignment

No new graded assignment this week. A4 (Bayesian analysis by grid) is due at the start of the Week 12 lecture. A5 (MCMC from scratch) is released in Week 12 and builds directly on today's `metropolis` function, so keep your lab notebook tidy.

Optional practice (ungraded): Exercise 2 (multiple starting points) is the best preparation for Week 12.

1.1.11 Further reading

- Kruschke (2014), chapter 7. Markov chain Monte Carlo: an intuitive introduction to the Metropolis algorithm (the "politician" example), its logic and representativeness of samples. *(Main reading.)*
- Farrell & Lewandowsky (2018), chapter 7. Bayesian parameter estimation with MCMC, applied to cognitive models.
- Martin, Kumar & Lao (2021): the sections on inference engines and MCMC [verify chapter].

Preview for Week 12: Gibbs sampling (no rejections at all), and the question every MCMC user must answer: has my chain converged? We will compute R-hat and the effective sample size ourselves, then check them against ArviZ.